

Q1 memoize

Closure

Design

O(1) time · O(n) space

```
function memoize(fn) {
  const cache = {};
  return function(x) {
    if (cache[x]) return cache[x];
    return cache[x] = fn(x);
  };
}
```

SAY: Closure holds cache. Check cache first, compute only on miss.

Q2 flatten

Array

Recursion

O(n) time · O(n) space

```
function flatten(arr) {
  return arr.reduce((acc, val) =>
    Array.isArray(val)
      ? acc.concat(flatten(val))
      : acc.concat(val)
    , []);
}
// [1, [2, [3]]] -> [1, 2, 3]
```

SAY: Recursion + reduce. If item is array recurse, else concat directly.

Q3 Closure — Counter

Closure

O(1) time · O(1) space

```
function outer() {
  let count = 0;
  return function inner() {
    count++;
    console.log(count);
  };
}
const counter = outer();
counter(); // 1
counter(); // 2
```

SAY: Inner function retains access to count in outer scope after outer returns.

Q4 throttle

Closure

Design

O(1) time · O(1) space

```
function throttle(fn, limit) {
  let flag = true;
  return function(...args) {
    if (flag) {
      fn.apply(this, args);
      flag = false;
      setTimeout(() => { flag = true; }, limit);
    }
  };
}
```

SAY: Flag blocks calls during cooldown. Fires immediately then enforces limit.

Q5 removeDuplicates

Array

Set

O(n) time · O(n) space

```
// Best approach using Set:
function removeDuplicates(arr) {
  return [...new Set(arr)];
}

// Your approach (O(n^2)):
function removeDuplicates(arr) {
  let result = [];
  for (let i = 0; i < arr.length; i++) {
    if (!result.includes(arr[i])) result.push(arr[i]);
  }
  return result;
}
```

SAY: Best: spread a Set. Mention Set approach is O(n), includes() loop is O(n^2).

Q6 groupAnagrams

String

HashMap

O(n*k log k) time · O(n*k) space

```
function groupAnagrams(strs) {
  const map = new Map();
  for (const s of strs) {
    const key = s.split('').sort().join('');
    if (!map.has(key)) map.set(key, []);
    map.get(key).push(s);
  }
  return [...map.values()];
}
```

SAY: Sort each string as canonical key. All anagrams share same sorted form.

Q7 debounce

Closure

Design

O(1) time · O(1) space

```
function debounce(fn, delay) {
  let timerId;
  return function(...args) {
    clearTimeout(timerId);
    timerId = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}
// fires only after 500ms silence
```

SAY: Each call resets timer. fn fires only after delay ms of silence.

Q8 isValid (Parentheses)

String

Stack

O(n) time · O(n) space

```
function isValid(s) {
  const stack = [];
  const map = {'(': '(', ')': ')', '[': '[', ']' : ']'};
  for (const c of s) {
    if ('(['.includes(c)) stack.push(c);
    else if (stack.pop() !== map[c]) return false;
  }
  return stack.length === 0;
}
// '()[]{}' -> false
```

SAY: Push openers. For closers — pop and verify match against map.

Q9 maxSubArray — Kadane's

Array

O(n) time · O(1) space

```
function maxSubArray(nums) {
  let curr = nums[0], max = nums[0];
  for (let i = 1; i < nums.length; i++) {
    curr = Math.max(nums[i], curr + nums[i]);
    max = Math.max(max, curr);
  }
  return max;
}
// binarySearch: O(log n) — halve search space
// longestSubstring: sliding window O(n)
```

SAY: Extend or restart? curr = max(num, curr+num). Kadane's greedy pass.